

南京大学《生成式软件工程》2026 秋季学期

第 2 讲：提示词工程

从一句话指令到可验证的上下文工程

主讲：蒋炎岩 中文精讲笔记



视频标题：提示词工程 [02-Raw/26 生成式软件工程/NJU]

视频作者/频道：绿导师原谅你了

发布日期：2026-09-01

时长：01:38:25

视频：<https://www.bilibili.com/video/BV1CQt365EzW/>

课程主页：<https://jyywiki.cn/GSE/2026/>

目录

1 导读：提示词不是魔法咒语	2
2 从 Prompt 到 Context：模型真正看见什么	3
3 立即行动：把 AI 变成学习反馈	5
4 上下文的分层：规则、工具与安全边界	6
5 什么叫完成：从 Intent 到 Spec 再到 Impl	9
6 约束不是形容词：从“AI 味”到设计语言	11
7 Token、思维链与 Verifier：浅步骤怎样串成深计算	13
8 注意力也是资源：规则何时常驻，何时加载	15
9 案例：把 AI Reviewer 从角色扮演升级为证据审计	17
10 从一次成功到 Long-Horizon：Skill、反馈与代码库	20
11 研究失败：Token 花完，知识为什么没有增加	22
12 在不确定性中搜索：算力、宽度与多样性	24
13 结语：把一句话升级为可验证系统	26

1 导读：提示词不是魔法咒语

本讲主问题

当 Agent 可以读取项目、调用工具并持续工作时，我们究竟在“提示”什么？答案不再是一句话，而是一套把意图、事实、权限、行动和验收组织起来的运行环境。本讲真正讨论的是：如何让模型在这个环境里持续向可验证结果推进。

很多提示词教程从句式开始：指定角色、补充背景、要求一步一步思考、给出输出格式。这些方法并非无效，但它们只看见了用户最后输入的那一小段文字。真实 Agent 在第一次回答之前，往往已经读过系统提示词、项目级规则、按需加载的 Skill、工具定义、权限策略、环境状态以及历史对话。用户消息只是其中靠近末端的一部分。

因此，本讲把问题从“怎样写一句更灵的 prompt”推进到“怎样构造一个模型可以可靠行动的 context”。这里的可靠不是语气更像专家，而是四件事同时成立：目标没有歧义，证据保持新鲜，动作边界可执行，完成条件可检查。只追求漂亮输出，会得到一次演示；把这四件事组织成闭环，才可能得到可重复的工程能力。

贯穿全讲的五条线索

1. **上下文是输入面**：模型实际响应整个 token 序列，而非孤立的一句话。
2. **验证器定义终点**：若系统无法识别成功，模型也无法稳定知道何时停止。
3. **约束压缩搜索空间**：每个未说明的选择都会由模型按默认偏好补齐。
4. **反馈创造长程能力**：尝试、检查、观察和修复组成的循环比一次“深思”更重要。
5. **人仍承担外部责任**：人定义问题、边界和后果，AI 在边界内扩大搜索与执行。

1.1 证据与整理方法

本笔记使用原视频、整段本地中英 ASR、每 15 秒抽取的 394 张完整画面，以及课程主页作为证据。Bilibili 页面未暴露字幕轨，因此字幕是本地转写并做保守术语修正的辅助时间轴，不被当作逐字稿。39 张插图均经过联系表筛选、全分辨率复核和字幕上下文核对；画面保留讲台与投影的原始布局，没有把讲者裁掉后冒充原课件。

阅读边界

讲义中的历史节点、论文案例、漏洞编号和实验数字，首先还原的是课堂陈述。除课程主页外，本笔记没有把这些内容重新包装成独立的外部事实核查。现场生成结果说明的是方法和失败模式，不代表某个模型、仓库或评审流程已经通过通用基准验证。

1.2 全讲路线图

视频区间	核心问题
00:00–10:30	Prompt 从哪里来; Agent 看见什么; AI 如何把第一次学习尝试变便宜
10:30–18:45	系统提示词、项目规则、Skill、工具、Harness、沙箱和人工批准如何分层
18:45–31:15	什么叫完成; 如何把 Intent 逐层压缩成 Spec 与可验收的 Impl
31:15–45:30	Token 怎样形成计算深度; 形式约束与 Verifier 能看到什么、看不到什么
45:30–54:45	长上下文中的注意力竞争; 规则该常驻还是按需加载
54:45–71:00	AI 论文审稿: 怎样从角色扮演升级为证据链和独立反证
71:00–84:45	Skill 蒸馏、Long-Horizon、Greenfield 与参考代码库的帮助/负迁移
84:45–98:25	研究失败、吞吐量、机制化记忆、不确定性搜索和多样化采样

1.3 本章小结

提示词工程不是寻找一句万能话术, 而是让模型持续看见正确的问题、可用的证据、合法的动作和明确的终点。后续所有例子都可以放回这四格框架中理解。

2 从 Prompt 到 Context: 模型真正看见什么

2.1 ChatGPT 普及了术语, 但没有发明它

课堂从 “prompt” 一词的历史开始。幻灯片把 GPT-2 (2019)、GPT-3 (2020) 以及 2021 年前后的 prompt programming、prompt-based learning 连在一起, 说明今天熟悉的提示词概念有连续演化过程。ChatGPT 的关键作用, 是把自然语言提示变成大众与模型交互的日常入口, 而不是凭空创造了这套思想。



图 1: Prompt 术语早于 ChatGPT: 从 GPT-2、GPT-3 到提示词编程 (原视频画面: 00:00:15–00:00:30)

聊天框容易制造一个错觉: 屏幕上看见的输入框就是模型的全部输入。对 Agent 而言, 真正的条件分布更接近幻灯片上的表达:

$$\Pr[\text{token} \mid \text{context}]$$

这里的 context 不仅包含用户文字，也包括系统角色、目标模式、持久规则、已加载的技能、工具调用及返回值。甚至“这段内容是指令、证据还是工具输出”的边界，也由 Harness 以消息类型和优先级提供给模型。

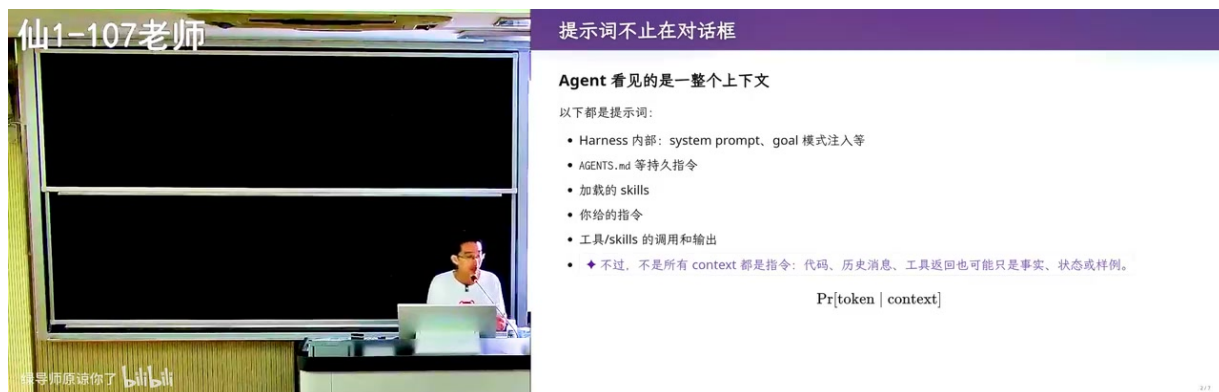


图 2: 提示词不只在对话框，Agent 看到的是整个运行上下文（原视频画面：00:01:30–00:01:45）

2.2 上下文工程管理的是工作集

上下文窗口有容量，却不等于拥有同样大小的有效注意力。把所有文档都塞进去，会同时引入过期事实、冲突指令、无关细节和来源不明的结论。课程给出的上下文工程目标可概括为四点：足够支撑当前动作、随状态更新、能回到来源、能被验证。

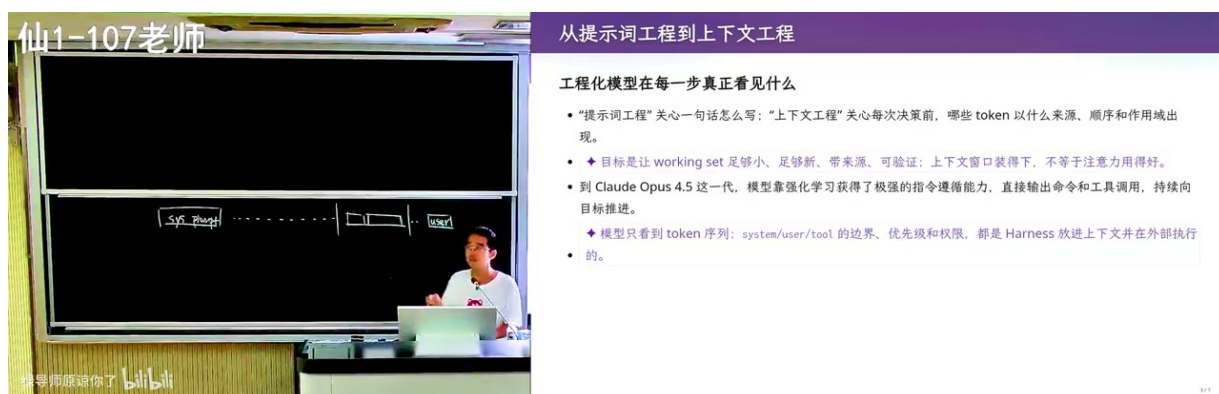


图 3: 从提示词工程转向上下文工程：组织足够、最新、有来源、可验证的工作集（原视频画面：00:03:00–00:03:15）

一次上下文盘点

在让 Agent 开始大任务前，列出六类输入：目标与交付物、权限与安全、工作流与工具、持久规则与风格、环境事实与证据、冲突时的优先级。然后追问：每一项从哪里来？何时更新？谁能覆盖？怎样证明模型真的遵守了？

“模型说它看见了”不是验证

让模型复述自己的上下文，只能得到一份候选解释。真正的验证要观察行为：它是否使用了规定语言，是否在权限边界请求批准，是否运行了指定测试，是否保留了不能覆盖的文件。报告

是计划，行为才是证据。

2.3 本章小结

从 prompt 到 context 的变化，是从句子优化转向系统设计。一个高质量上下文不追求最长，而追求对当前动作最有用、冲突最少、来源最清楚、验证最直接。

3 立即行动：把 AI 变成学习反馈

3.1 第一次尝试变得廉价

AI 最直接的教育价值，不是替学生交卷，而是降低“先试一下”的成本。过去，一个不确定的问题可能因为环境配置、资料检索和入门代码而被拖延；现在可以立刻构造一个最小例子，让 Agent 解释、实现或运行，再用结果校正自己的理解。课程把这种变化称为行动飞轮：问题越小，反馈越快；反馈越快，下一次提问越具体。



图 4: AI 降低第一次尝试成本：从最小问题开始转动反馈飞轮（原视频画面：00:04:45–00:05:00）

这并不意味着把任务整包外包。若学习者只收集模型答案，他获得的是完成感，不是能力。有效的循环应当保留自己的预测：先说出你认为会发生什么，再让系统运行；观察差异后，解释为什么错；最后改变一个条件重新测试。没有预测的实验很难暴露认知缺口，没有复测的解释也很难形成稳定模型。

3.2 费曼学习法：让 AI 当听众

课程把费曼学习法反过来使用：AI 不只是老师，也可以是耐心的听众。学习者选一个概念，用自己的话解释，让 AI 追问不清楚的地方，再回到教材、程序或实验核实。关键动作是“复述”，因为复制原文可以掩盖理解缺口，而面对追问重新组织因果关系会暴露断点。

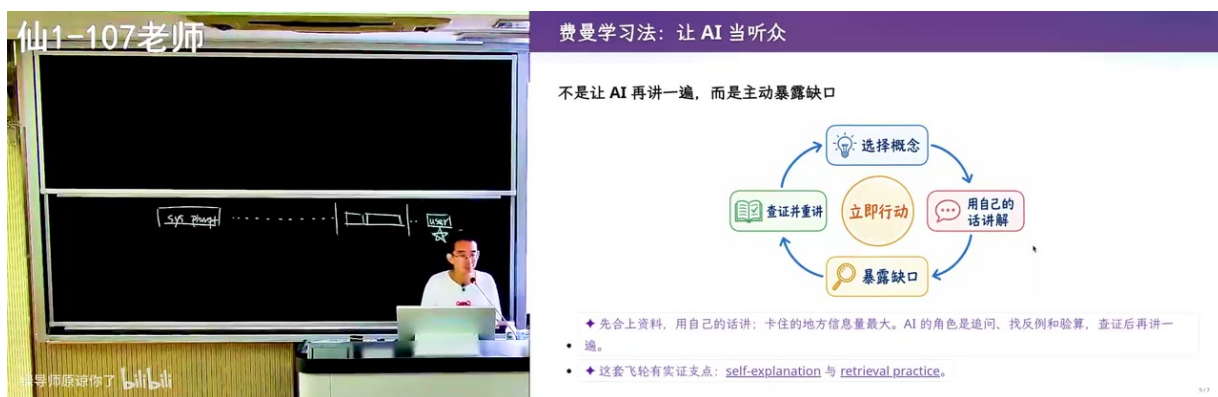


图 5: 让 AI 当听众：复述、暴露缺口、核实、重新讲授（原视频画面：00:09:30-00:09:45）

五分钟学习闭环

1. 写下一个足够小、可以在五分钟内验证的问题；
2. 先给出自己的预测和理由；
3. 让 AI 提供最小解释或实验，不要一次扩成完整教程；
4. 用代码、教材或反例核对，标出预测与结果的差异；
5. 不看原回答，重新讲给 AI，并要求它只追问最薄弱的一步。

学习证据发生了什么变化

“答案正确”在 AI 时代越来越弱。更有力的证据是：能否解释关键决策，能否预测变体，能否构造反例，能否在约束变化后修正实现。AI 可以参与每一步，但不能替学习者拥有这些可迁移的判断。

3.3 本章小结

AI 让尝试变便宜，但学习仍由反馈质量决定。把模型当作听众、实验搭档和反例生成器，比把它当作答案仓库更能形成长期能力。

4 上下文的分层：规则、工具与安全边界

4.1 先解剖，再优化

当 Agent 行为异常时，最有效的第一步往往不是再补一句强硬提示，而是盘点它已经看见什么。课程现场让 Agent 把上下文分成目标、权限、 workflow、规则、环境事实和优先级，并展示项目级 AGENTS.md、按需加载的 Skill、工具 schema 与会话历史如何逐层进入上下文。

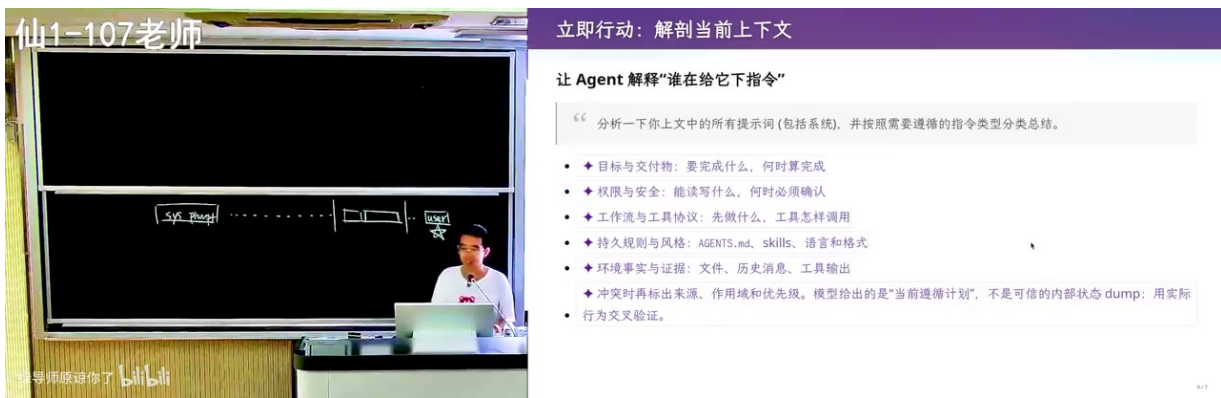


图 6: 解剖当前上下文：目标、权限、流程、规则、事实与冲突优先级 (原视频画面：00:11:00–00:11:15)

Skill 的意义不只是储存一段更长的 prompt。索引可以常驻，只在任务匹配时加载完整说明；任务结束后，细节可以离开活跃上下文。这样既保留可复用方法，又避免每个任务都为无关说明支付注意力成本。

4.2 谁判断命令安全

课程用 Codex 代码和安全案例回答了一个容易混淆的问题：模型可以提出命令，但它不是最终安全边界。Harness 解析动作、匹配批准策略并决定放行、询问或拒绝；操作系统沙箱和权限再提供不可绕过的强制约束。模型判断是有用信号，却不能替代执行层隔离。

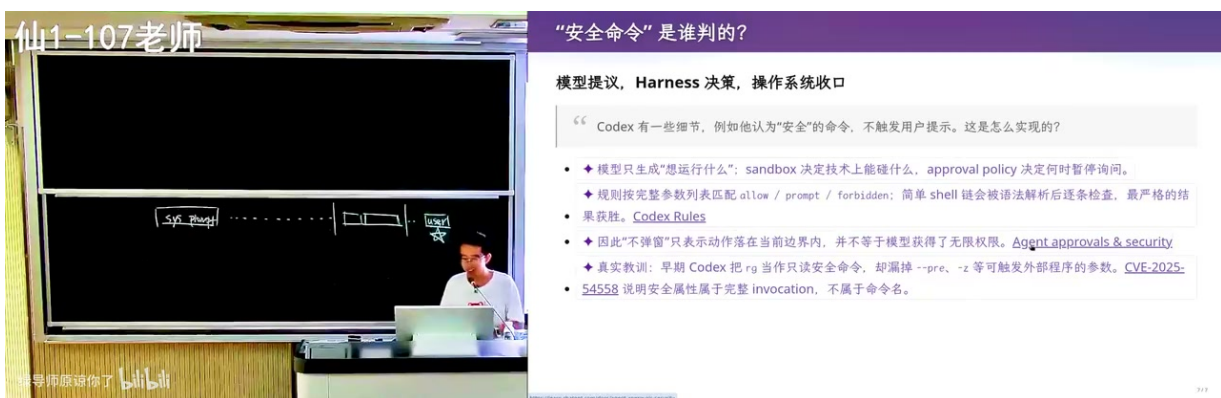


图 7: 模型建议、Harness 决策、操作系统强制：安全命令的三层责任 (原视频画面：00:12:30–00:12:45)

现场进一步展示了 Guardian 的分层思路：低风险且语法明确的动作可以走静态路由；边界不清的动作交给专门模型评审；高风险或语义仍不确定的动作请求人工批准。课程提到 CVE-2025-54558，意在说明调用边界和参数解析本身就是安全设计的一部分，而不是用一个漏洞证明所有 Agent 都不安全。

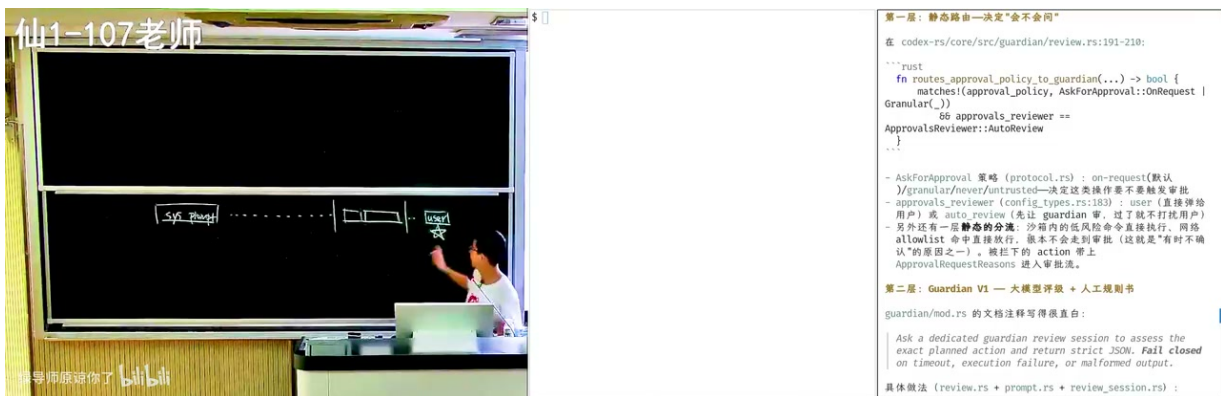


图 8: Guardian 的静态路由、模型评审与人工批准分层（原视频画面：00:17:15-00:17:30）

Fail-closed 的动作门

一个可靠动作门至少回答五个问题：动作解析后到底是什么？目标资源是谁？当前主体有什么权限？失败后能否恢复？无法判断时默认发生什么？最后一个问题尤其重要：不确定不应被解释为允许。

不要把礼貌拒绝当作安全

模型在文本中说“我不会执行危险命令”，并不能建立安全边界。提示词可能被覆盖，语义可能被误判，工具调用也可能把危险藏在参数里。真正的边界应位于模型之外，并保留可审计的批准理由与执行结果。

4.3 常驻规则与按需能力

稳定的组织约束适合常驻，例如语言、禁止覆盖用户文件、提交前必须运行测试。任务特有的方法更适合按需加载，例如如何制作课件、怎样审稿、怎样部署某类应用。工具返回值则是短期证据：应在状态变化后更新，而不是作为永远正确的记忆保留。

上下文层	生命周期	适合内容
系统/组织规则	长期	权限边界、沟通风格、不可违反的安全约束
项目规则	项目期	构建命令、目录边界、代码约定、验收门
Skill	任务期	可复用流程、判断标准、检查点、失败转向
会话证据	短期	当前文件、日志、工具输出、临时决定
用户动作	当前轮	目标变化、授权范围、优先级调整

4.4 本章小结

上下文分层同时解决两个问题：让正确规则在需要时出现，让真正的安全边界留在模型之外。写更多提示词不是目标，建立可解释的控制面才是。

5 什么叫完成：从 Intent 到 Spec 再到 Impl

5.1 模型为什么会提前交卷

语言模型擅长继续一段看起来像答案的文本，却不会天然知道真实系统何时完成。若上下文只出现“继续努力”“做到最好”，模型可能在输出结构看起来完整时结束；若上下文含有可观察的成功谓词，它就能把每次尝试同终点比较。课程把完成条件压缩为：

$$V(T) = 1$$

其中 T 是当前产物或轨迹， V 是验收器。公式并不承诺所有价值都能化成布尔值，它强调的是：凡是能明确检查的要求，都不要只留在人的愿望里。

图 9：上下文必须描述“完成”是什么，模型才有稳定的停止信号（原视频画面：00:19:00–00:19:15）

5.2 把愿望改写成验收条件

“做一套好用的导航”无法直接判定；“在指定误差内测得经度，并通过独立海试”则提供了证据门槛。课程以 1714 年英国《经度法》说明，工程进步常来自把宏大愿望改写为可验证结果：不是规定每一步怎样做，而是定义什么结果值得奖励。

图 10：把含糊愿望改写为可运行、可观察、可判定的验收条件（原视频画面：00:21:00–00:21:15）

把“好”改写成检查口

先保留意图，再补三类证据：功能证据回答“能不能做”；约束证据回答“是否在权限、性能和格式边界内”；使用证据回答“真实用户能否完成关键任务”。若某项暂时不能自动化，就明确写成需要人工复核，而不是假装它已经可验证。

5.3 Intent、Spec、Impl 的逐级放大

一句“生成极品飞车”隐藏了镜头、操控、地图、美术、碰撞、胜负、资产许可和性能目标。Intent 只表达想要什么体验；Spec 决定哪些行为和约束必须成立；Impl 才选择数据结构、引擎、算法与素材。每省略一层，模型就必须自行补齐更多默认值，候选空间近似按各选择维度相乘地扩大。

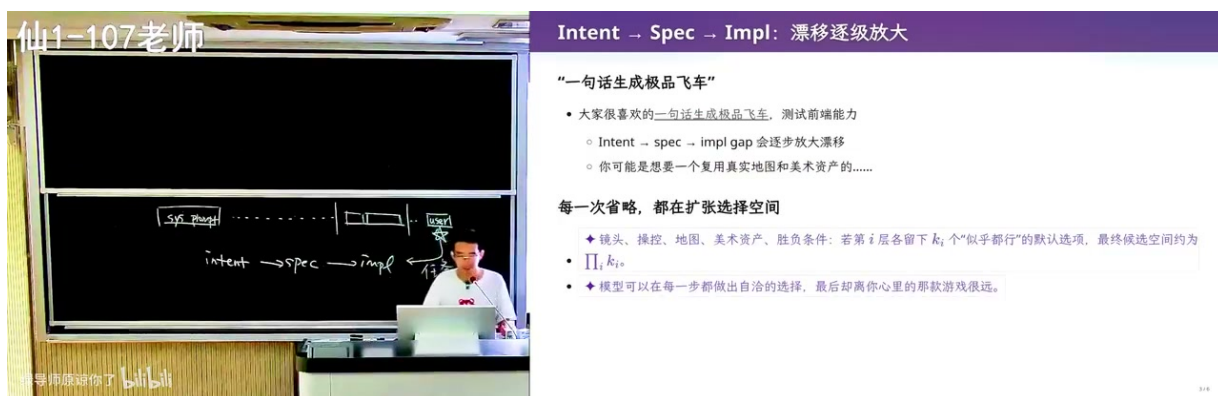


图 11: Intent 到 Spec 再到 Impl：每一次省略都在扩张选择空间（原视频画面：00:22:30–00:22:45）



图 12: 一句话生成驾驶游戏的现场结果：能运行不等于命中了真实意图（原视频画面：00:22:45–00:23:00）

课堂给出的 Rule of Thumb 是：写清指令，并承担所有“不明确”的后果。这不是要求写无限长的文档，而是要求选择性地消除高风险歧义。低价值细节可以授权模型决定；会改变产品身份、法律责任、数据安全或验收结果的细节，必须由人明确。

幻灯片以 1854 年 Raglan 的战场命令为反例：发令者默认骑兵与自己拥有相同视野，省略了“那些炮”究竟指什么，模糊处最终由执行者补全。这个例子把“歧义”从语言瑕疵提升为工程风险：执行能力越强，错误默认值造成的后果越大。



图 13: 要么完成，要么显式失败：不应以道歉或含糊说明代替证据（原视频画面：00:24:30–00:24:45）

把失败写进契约

好的任务契约不仅描述成功，也描述失败：运行哪些检查；检查失败后最多重试几次；缺少证据时如何报告；哪些情况必须停止并请求授权。这样“未完成”成为可接受、可诊断的状态，而不是被模型用漂亮措辞掩盖。

5.4 本章小结

Intent 决定方向，Spec 压缩选择空间，Impl 提供可运行产物，Verifier 决定是否到达。提示词工程真正困难的部分，是知道哪些自由度可以授权、哪些必须被规格和证据锁定。

6 约束不是形容词：从“AI 味”到设计语言

6.1 抽象形容词不能排除模板

“现代、简洁、有科技感”看似具体，实际上几乎没有排除任何模板。模型可以合法地生成渐变标题、玻璃卡片、深色背景，也可以选择完全不同的布局；结果都能自称“现代”。更好的提示不是堆更多风格词，而是指定可组合的设计语言：色彩范围、字体层级、栅格、留白、组件参考、交互状态和禁用模式。

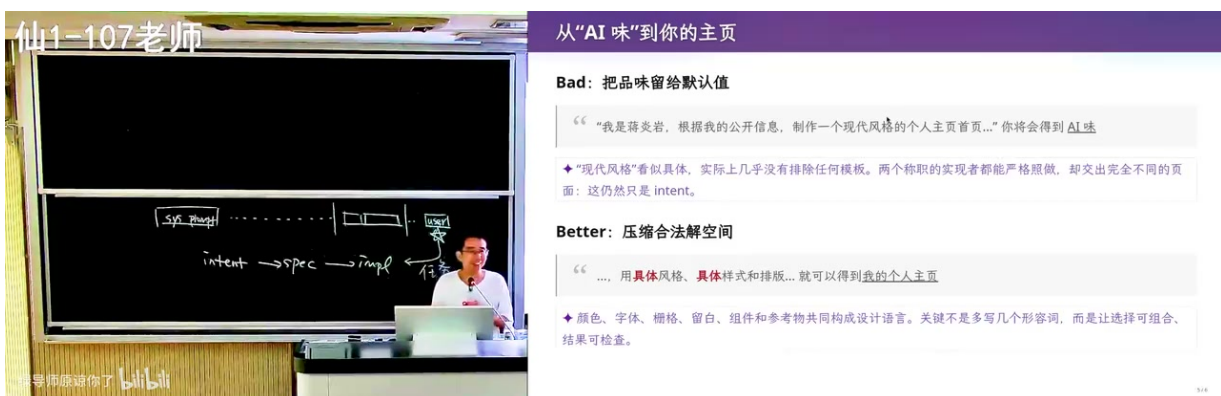


图 14: Bad 与 Better：抽象风格词对比可组合、可复核的设计约束（原视频画面：00:25:30–00:25:45）

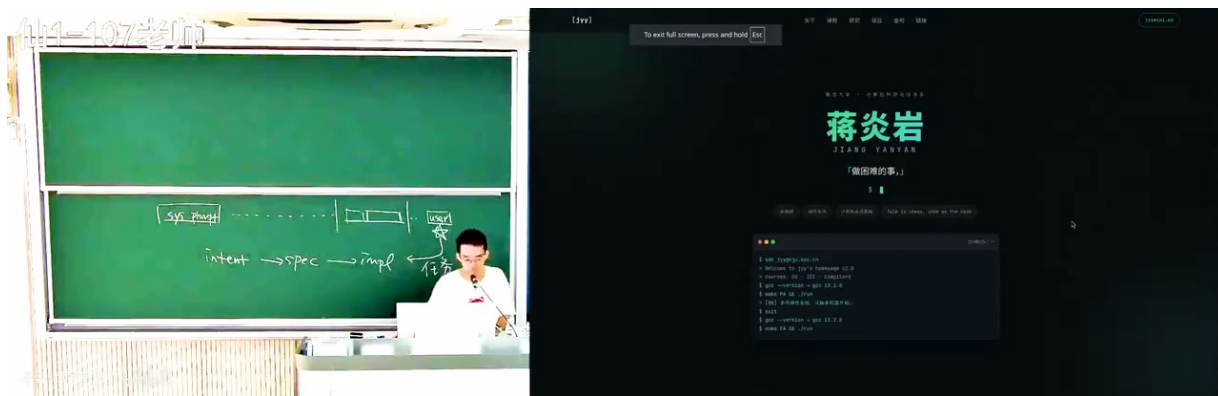


图 15: 在具体风格、排版与组件约束下生成的个人主页（原视频画面：00:26:15–00:26:30）

现场主页不是一个通用设计基准，它展示的是“合法解空间”概念。若约束只说个人主页，任何姓名、栏目和视觉系统都可能合规；若给出内容来源、参考组件与截图验收，模型就更容易在可接受空间内搜索。可验证设计仍需要真实浏览器渲染、不同视口、可访问性和内容准确性检查。

把形容词翻译成设计变量

把“专业”拆成信息层级、对齐方式、密度和内容证据；把“现代”拆成字体、色彩、间距、动效和组件语言；把“好用”拆成关键任务、状态反馈、错误恢复和可访问性。每个变量都应有参考、范围或截图检查口。

6.2 人定义问题，AI 搜索实现

课程提出新的分工：人更靠近 Intent 和 Spec，AI 更擅长在实现空间里搜索。实现成本下降后，真正稀缺的是对产品领域的认识、对用户后果的判断，以及知道什么不能交给默认值。Brooks 在 1986 年讨论软件的 essence 与 accident；课程借此强调，编码偶然性变便宜，并不会让需求、规格和设计概念结构消失。

新分工：人定义问题，AI 搜索实现

Intent → Spec → Impl

- 明确你的意图，需要你对**产品领域**的知识
- 把意图变成设计，需要你对**软件工程**的知识
- 实现反而多数可以交给 AI，已经超过中级程序员了
- ✦ 可以把边界画在“搜索”和“判定”之间：人定义目标 G 与验收器 V ，AI 搜索实现 x ，直到 $V(x) = 1$ 。

Impl 变便宜，判断没有

- ✦ Brooks 在 1986 年区分了软件的 essence 与 accident：困难在于构造、规格化、设计和测试概念结构，而不只是把它表示成代码。[No Silver Bullet 原始技术报告](#)
- ✦ 实现可以外包：验收标准和失败责任不能一起外包。否则 AI 优化的是它猜到的代理目标，不是你的产品目标。

图 16: 人定义问题和验收目标，AI 在实现空间持续搜索（原视频画面：00:31:15–00:31:30）

“AI 搜索实现”不等于人只写需求

实现过程会暴露新约束，Spec 也会被实验修正。人的工作不是在起点写完一份完美文档后退出，而是维护目标、吸收证据、决定取舍，并对系统进入真实世界后的后果负责。

6.3 本章小结

提示质量的核心不是辞藻，而是选择空间。把审美和产品意图翻译成可组合变量、参考物和验收口，才能让模型的实现能力服务于真实目标。

7 Token、思维链与 Verifier：浅步骤怎样串成深计算

7.1 一步很浅，一串可以很深

模型生成每个 token 时只做一次有限前向计算，但新 token 会进入后续 context，成为下一步可读取的中间结果。课程把它类比为图灵机纸带：单步转移简单，中间状态被写下后，一串步骤可以积累计算深度。这解释了为何 test-time scaling 有效：不是一句提示突然提升参数，而是给模型更多建立、检查和修正中间状态的机会。



图 17：单步 token 计算有限，但中间结果进入上下文后可以串成深推理（原视频画面：00:33:15–00:33:30）

多写并不自动等于多想。只有当中间状态能被后续步骤使用，并且存在反馈信号时，额外 token 才可能带来改进。没有检查口的长思维链可能只把最初假设写得更完整；有检查器的短循环则可以不断把错误拉回。

计算深度来自可复用中间状态

有效中间状态应当让下一步更容易判断：当前假设是什么，已有证据是什么，哪项检查失败，下一次只改变什么。若一段推理没有留下这些结构，它对后续 token 的价值就接近叙述，而非计算。

7.2 形式约束把生成变成搜索

课程用藏头诗、数字顺序、字数、部首和同音故事展示 Verifier 的力量。开放写作本来有巨大空间；一旦约束可以程序检查，模型就能先生成候选，再根据失败位置继续补偿。它不必一次命中，而可以在上下文中保留“哪一条没过”的局部反馈。

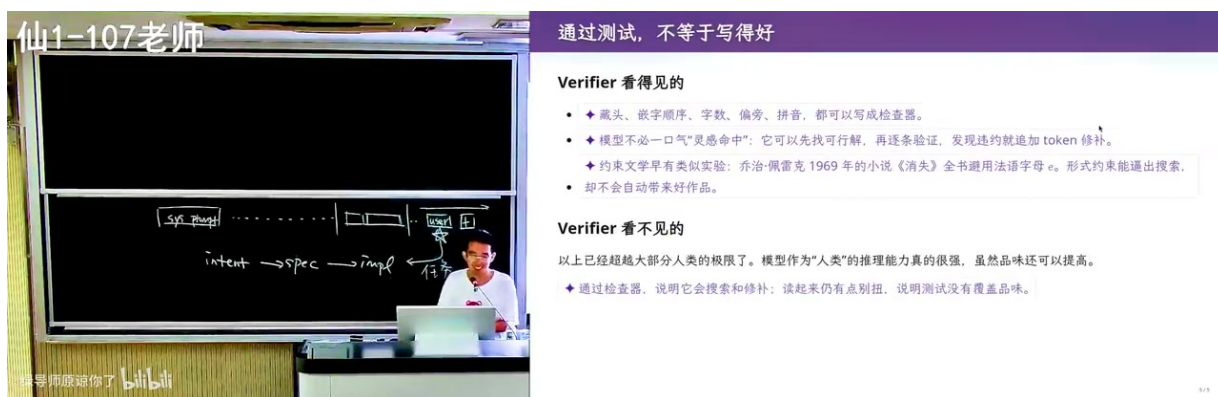


图 21: Verifier 看得见形式约束，却未必看得见趣味、论证与品味（原视频画面：00:44:30–00:44:45）

测试套件是代理目标

模型被训练成执着满足检查时，也可能找到测试的漏洞。测试覆盖表面行为，不能证明用户目标全部满足。需要保留隐藏测试、独立评审、真实使用情境和反例构造，防止“分数被刷满，目标被绕开”。

双层 Verifier

第一层使用机器可判定检查，快速排除格式、类型、边界、权限和回归错误；第二层由独立人或独立路线检查解释、审美、证据充分性和真实用户后果。两层结论冲突时，不要用第一层 PASS 覆盖第二层问题。

7.4 本章小结

Token 提供连续试错的计算载体，Verifier 提供方向。真正的能力来自两者耦合：中间状态能被复用，错误能被定位，结果能被独立检查，同时承认检查器永远有盲区。

8 注意力也是资源：规则何时常驻，何时加载

8.1 过去一直在场，但不保证一直被用好

Self-attention 允许每个新 token 重新读取过去上下文。全局的“用中文回答”可能一直影响输出；局部的代码错误也可能在总结阶段重新被注意。然而“可见”不等于“可靠利用”。当模型专注于实现一个子任务时，邻近代码、工具输出和当前错误会占据更强注意力，早期目标和远处约束可能被弱化。

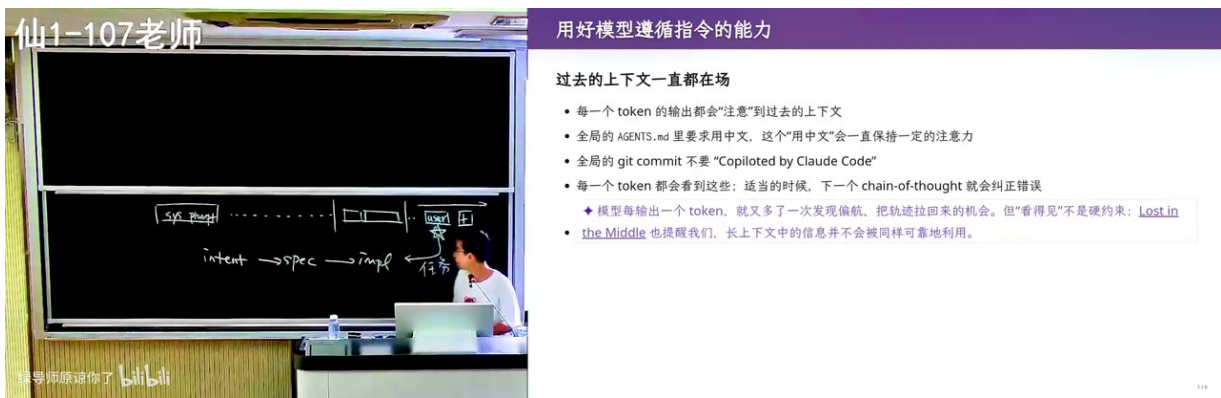


图 22: 过去的上下文一直可见，但每次 token 对旧信息的注意并不相同（原视频画面：00:47:30–00:47:45）

这解释了阶段边界为何重要。实现结束后先做一次“呼吸”：重述目标、记录已证实事实、列出未解决问题，再进入下一阶段。总结不是为了写漂亮报告，而是把关键状态搬到注意力更有利的位置。

8.2 规则会纠偏，也会抢注意力

把每次失败都追加进全局规则，会逐渐形成厚重、互相防御的提示墙。规则越多，冲突越难发现，模型在每一步都需要分配注意力处理无关约束。相反，完全不保留经验又会重复踩坑。课程给出的方向是分层：稳定原则常驻，任务方法放 Skill，细节通过测试和接口固化。

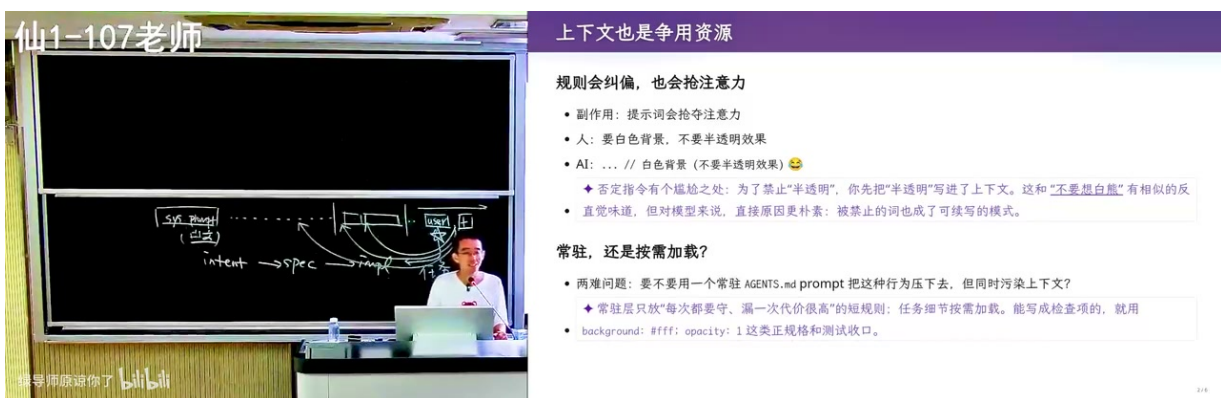


图 23: 上下文既是纠偏资源，也是竞争注意力的资源（原视频画面：00:50:45–00:51:00）

把“记住”升级为“机制”

关键约束不要只写在 prompt：类型约束交给编译器，接口约束交给 property tests，回归约束交给 CI，权限边界交给执行层，阶段状态交给 checkpoint。机制可以失败并留下证据；纯文本提醒失败时，模型常常仍会声称自己遵守了。

问题	只追加 prompt	机制化做法
输出格式漂移	再强调一次格式	schema 校验并返回具体字段错误
回归反复出现	写“不要破坏旧功能”	固化回归测试与 CI gate
权限越界	写“请小心”	沙箱、allowlist、人工批准
长任务忘记目标	重复粘贴完整历史	checkpoint 保存目标、证据和未决问题
方法难以复用	保存上次最终答案	Skill 保存搜索策略、判断标准和转向条件

8.3 本章小结

上下文窗口是容量，不是保证。通过分层、阶段重述和机制化检查，把最关键的信息放在最能影响下一步的位置，才是长任务中的记忆工程。

9 案例：把 AI Reviewer 从角色扮演升级为证据审计

9.1 “你是审稿人”只规定了表面角色

课程现场把自己的论文交给 AI，要求模拟 ICSE 审稿。简单角色提示容易学到审稿格式、语气和常见栏目，却不保证模型执行了真正的证据程序。它可能复述摘要、罗列一般性优缺点，再给出看似合理的分数；这种输出像 review，却没有建立 claim 到 evidence 的链条。

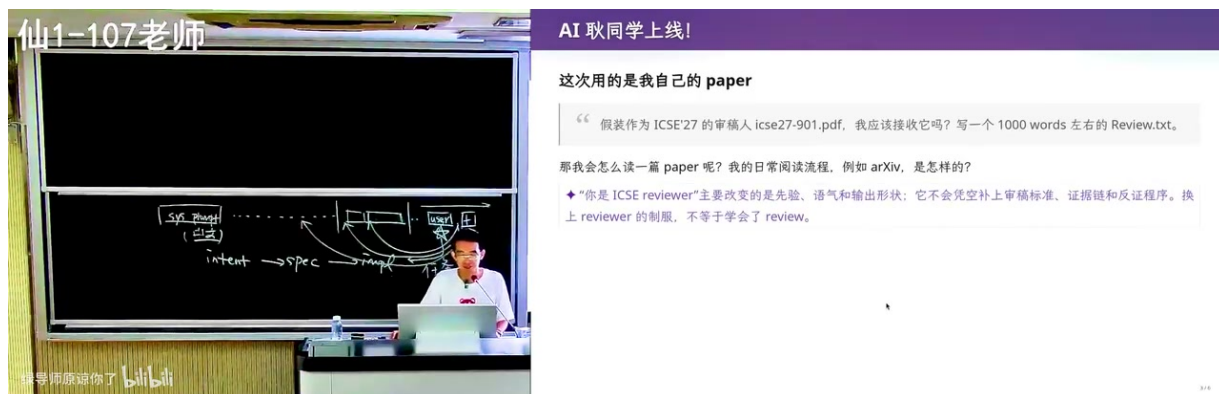


图 24: 让 AI 审自己的论文：角色提示不等于掌握审稿方法（原视频画面：00:55:00–00:55:15）

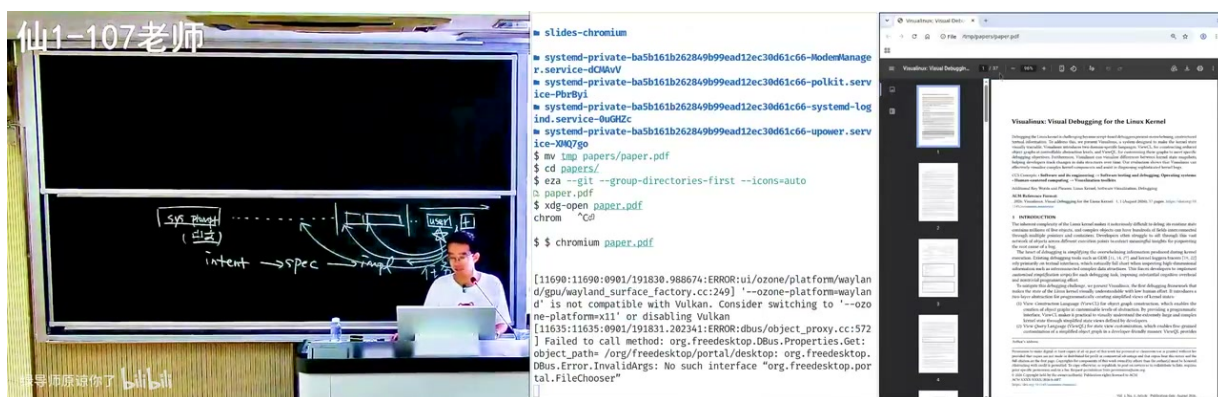


图 25: 现场读取论文并生成审稿意见: 可观察轨迹比最终语气更重要 (原视频画面: 00:55:45-00:56:00)

更可靠的做法是先提供审稿方法: 论文声称的主要贡献是什么; 每个核心 claim 需要什么证据; 实验究竟排除了哪些替代解释; 哪些结论超出数据覆盖; 若站在反方, 最小反例是什么。身份可以帮助设定受众和格式, 方法才决定推理路径。

9.2 显式建立 claim-evidence-assumption

现场演示把论文断言、讲者记忆和置信度放入表格。表格的价值不是让 AI “相信作者”, 而是暴露证据层级: 哪些事实来自论文, 哪些来自已有知识, 哪些只是需要进一步查证的记忆。对于论文外部事实, 应回到原始来源; 对于实验结论, 应迫到表格、脚本和数据, 而不是停在摘要措辞。

论文断言	我的记忆	置信度
StackRot (CVE-2023-3269): mmap 读锁下退避 maple tree 节点 → RCU 延迟回收 → 读者侧 UAF	与我的记忆一致: expand_stack 路径持续操作 maple tree, 修复方向是保持写锁。叙述可信	高
Dirty Pipe (CVE-2022-0847): copy_page_to_iter_pipe() 未初始化 flags, 残留 CAN_MERGE	或可一致 (可验证可信)	高
Dirty Pipe "present since 4.9. 5.8 后才可用"	可疑。我的记忆是: CAN_MERGE 机制随 5.8 引入, 漏洞自 5.8 起就存在: 4.9 只是修复旧 stable 分支 (5.16.11/5.13.25/5.10.182/5.4/4.19/4.9.301)。作者说可能把“即补范围”误当成了“引入时间”	中低
maple tree 由 Linux 6.1 引入	I	正确
Linux 6.1 的		高

Working...

/tmp/papers
*36k 46.7k R164k \$0.007 4.0%/1.0M (auto)

deepseek-v4-Flash • high

图 26: 把论文断言、已有记忆与置信度对齐, 显式暴露证据缺口 (原视频画面: 01:00:30-01:00:45)

一条 claim 的审计卡

1. 原文到底声称什么, 范围与条件是什么;
2. 需要哪类证据才能支持;
3. 论文实际提供了什么, 证据位于何处;
4. 哪些替代解释没有排除;
5. 什么观察会推翻或显著削弱结论;
6. 当前结论的置信度以及仍需补查的来源。

9.3 三条独立路线先分离，再汇合

课程建议把审稿拆成三条相对独立的路线。第一条剥离贡献：先不看作者包装，重写问题、方法和新增信息；第二条证据审计：为每个核心 claim 建账本；第三条反方复现：在最弱或最反直觉的位置重算结果、寻找最小反例。三条路线独立形成判断后再汇合，可以减少多个 reviewer 只是重复同一锚点的风险。

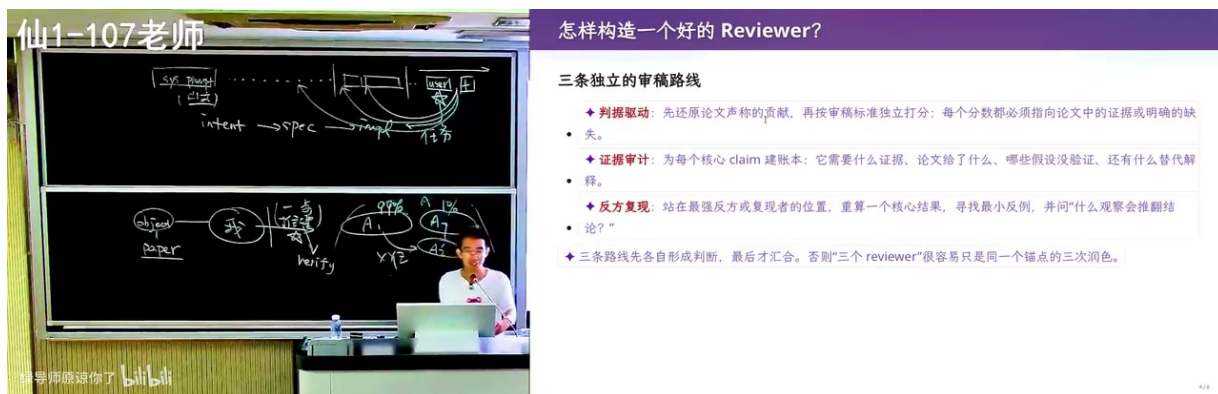


图 27: 剥离贡献、证据审计、反方复现三条独立审稿路线（原视频画面：01:09:45–01:10:00）

9.4 不要被作者的方法论带着走

论文信息密度很高，顺序本身会分配注意力。若 reviewer 从标题、摘要、相关工作一路顺读，作者的术语和因果结构会成为默认框架。课程提醒，critical thinking 可能在不知不觉中变成“在作者框架里挑小问题”。更强的做法是在深读前独立写下：它声称解决什么、最强替代解释是什么、什么结果会推翻结论；再回到正文寻找证据。

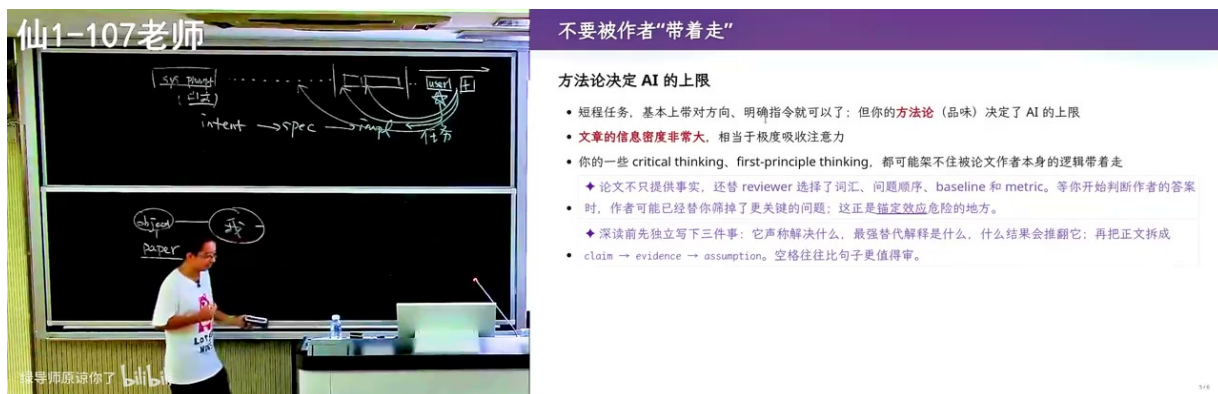


图 28: 避免被作者结构锚定：先独立重建 claim、evidence 与 assumption（原视频画面：01:11:00–01:11:15）

AI 审稿的可靠边界

模型可以扩大检索、整理和反例生成，但不能凭流畅语气补全缺失实验。若没有论文、补充材料、代码和数据，它只能评审可见证据。若外部事实没有原始来源，必须标为待核实，而不是利用模型记忆给出确定结论。

9.5 本章小结

高质量 Reviewer 不是一种角色语气，而是一套可审计程序。把贡献、证据、假设和反例分离，把不确定性写出来，AI 才能成为研究判断的放大器而不是结论生成器。

10 从一次成功到 Long-Horizon: Skill、反馈与代码库

10.1 把品味蒸馏成 Skill

当一次任务成功时，最值得保存的不是最终答案，而是通向答案的路线：如何拆解，先看什么证据，用什么判断标准，在哪些检查点停下来，失败后怎样换路。课程称之为把“品味”蒸馏成 Skill。Skill 本身也应形成闭环，避免长指令中后半段越来越难遵循。

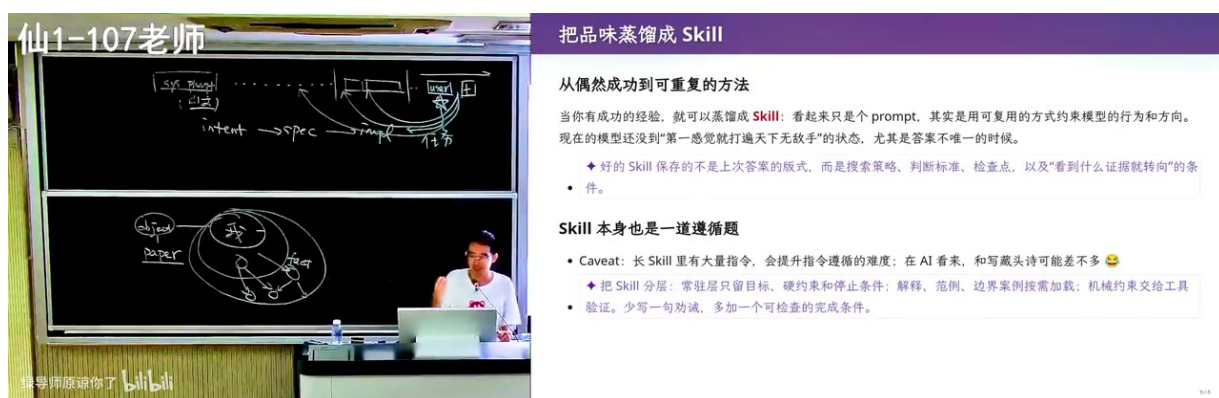


图 29：从偶然成功到可重复方法：保存搜索策略、判断标准与转向条件（原视频画面：01:15:00–01:15:15）

Skill 应保存什么

保存问题分解、证据路由、判断门、验证方法和失败转向；少保存某次任务的固定答案。把长 Skill 分成常驻目标、任务阶段和按需参考，让每一阶段都有可观察的完成条件。

10.2 长程能力是反馈循环训练出来的

Long-Horizon 任务包含未知：起点只有大致 Intent, Spec 和 Impl 都会在探索中变化。模型之所以能持续写代码，并非一次规划覆盖了全部未来，而是代码世界提供密集反馈：构建、测试、benchmark、运行日志和界面截图不断告诉系统下一步是否更接近目标。

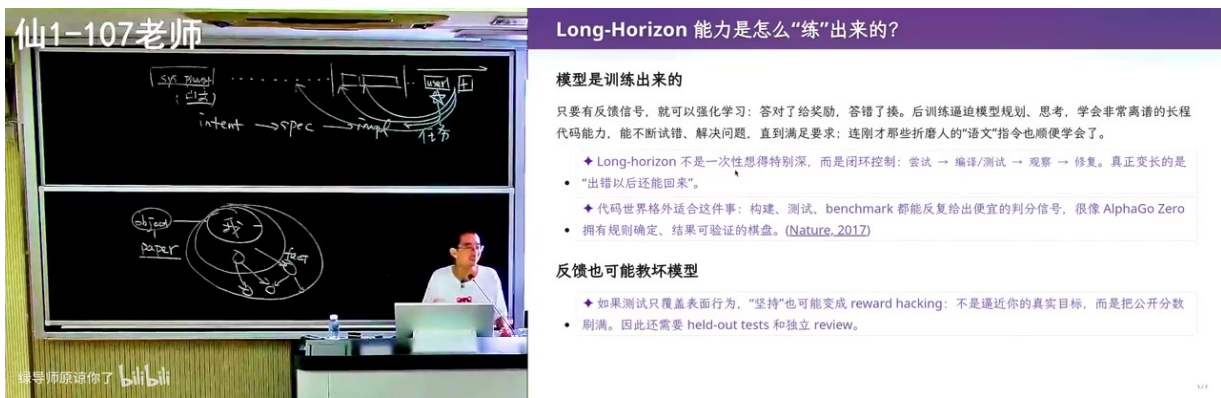


图 30: 尝试、编译/测试、观察、修复：反馈循环形成长程能力（原视频画面：01:15:45–01:16:00）

反馈也可能教坏模型。若测试只覆盖表面行为，Agent 会把“坚持”变成 reward hacking：反复修改直到分数通过，却绕开真实目标。因此还要保留 held-out tests、独立 review 和不由执行者控制的验收证据。

10.3 Greenfield 为什么未必更容易

从零开始的代码很少，看似上下文干净；但所有命名、分层、错误处理、测试和演化边界都尚未决定，选择空间反而最大。成熟仓库含有大量代码，也含有可供 in-context learning 的工程惯例。问题不在字节多少，而在信息密度：哪些代码表达稳定设计，哪些只是历史事故。



图 31: Greenfield 代码少不等于信息少；成熟仓库提供惯例也积累噪声（原视频画面：01:17:00–01:17:15）

课程讨论了“给 AI 一个相近优质项目是否更可靠”的猜想，并引用 RepoCoder、CodeRAG-Bench 等研究线索。幻灯片提到 RepoCoder 的迭代检索相对当前文件基线提升超过 10%，同时明确保留边界：这支持“好示例可能有用”，却没有证明外来仓库对所有项目都有益；外部有效性、负迁移和检索到错误片段的问题仍未解决。因此，更有价值的是设计对照实验，而不是把“参考仓库越大越好”当成定律。

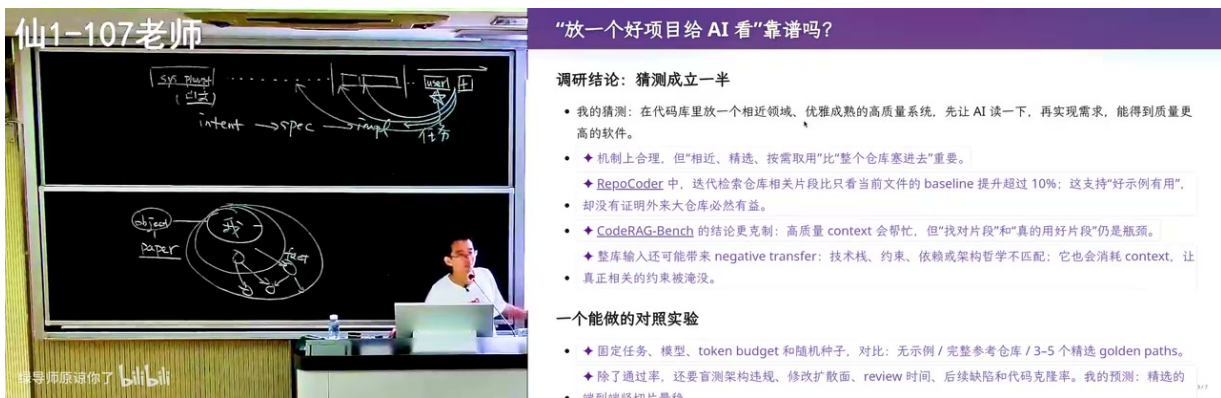


图 32: 用固定任务、模型和预算检验参考仓库的帮助与负迁移 (原视频画面: 01:21:00–01:21:15)

一个可执行的仓库复用对照

固定任务、模型、token 预算和随机种子，比较三组上下文：无示例；完整参考仓库；人工精选的 3–5 条 golden paths。记录测试通过率、首次架构违规、返工轮次、review 时间和后续缺陷。只有这样，才能区分“内容更多”与“真正提供了高质量惯例”。

10.4 本章小结

Long-Horizon 的核心不是让模型连续运行更久，而是让每一阶段都产生可归因反馈。Skill 保存方法，代码库提供惯例，测试与独立评审防止反馈被投机利用。

11 研究失败：Token 花完，知识为什么没有增加

11.1 总分不是可用反馈

课程讲述一个符号执行引擎优化案例：Agent 可以不断尝试 benchmark、修改实现、再跑一次，花费大量 token 却没有形成新知识。问题不一定是模型不够聪明，而是反馈太昂贵、太局部不可见、太难归因。一个总 benchmark 分数几乎不告诉系统：慢在路径爆炸、求解器、缓存、内存、搜索策略，还是实现 bug 与环境噪声。



图 33: Research Failure: token 花完但知识没有增加，原因是缺少局部可归因反馈 (原视频画面: 01:25:15–01:25:30)

研究允许“原始想法不成立”成为最佳结论，但 Agent 往往被训练成必须完成任务。没有可证伪假设时，它可能把错误路线包装得越来越完整。课程提到 KLEE 对搜索策略、约束缓存和独立约束集等机制的拆分，强调应先把大问题分成可测机制，再逐项建立几十秒内返回的 micro-benchmark。

Research Failure 的正确产物

一次失败至少应增加一种知识：哪类输入触发问题，哪个机制被排除，哪个指标与目标无关，什么实验能区分两个假设。若只留下“跑了很多 token，没有提升”，过程就没有形成可复用证据。

11.2 少干预，反而做得更多

人盯着每一步，会优化单个任务的 latency：看到偏航立即纠正，当前任务可能更快。但全天生产力关心 throughput：多个任务能否并行，失败能否恢复，结果能否稍后 review。只要任务边界清楚、失败可恢复，放手让 Agent 跑，再按检查口批量审阅，往往比持续干预更高效。

仙1-107老师

少干预，反而做得更多

Latency 和 Throughput 是两回事

一般来说，我对要解决的问题了解更多，所以我干预，会完成得更快。但是我越来越少干预了：AI 走路也能完成，那我还不如切换到另一个任务。只要结果能满足，我更倾向于不管。

- ◆ 人类干预优化单个任务的 latency：放手让 Agent 跑，优化的是一整天的 throughput。机器多走一点弯路，换回最稀缺的人类注意力。
- ◆ 放手成立有两个前提：任务可以并行，失败可以便宜地恢复。否则“少干预”只是把故障推迟。

Review 什么？

- 比如接口我 review 过了，实现是 AI slop，最后出了问题了修就行 😊
- ◆ 按“不可逆性 × 爆炸半径”分配注意力：接口、不变量、数据格式、权限和测试早看；可替换的叶子实现可以晚看。
- 丰田的“自働化”也是这个逻辑：一个人照看多台机器，靠异常自动停机 and on 报警，而不是盯住每一步。Agent 的对应物是 CI、预算、权限边界和 stop condition。([Toyota Production System](#))

图 34: 区分单任务延迟与全天吞吐量：少干预的前提是并行和可恢复 (原视频画面：01:28:45–01:29:00)

“少干预”不是不 review。Review 的重点应从观察每个 token，转向检查接口是否稳定、权限是否越界、数据格式是否正确、测试是否可信、失败能否回滚。丰田式自动化的启发是让异常触发停止和告警，而不是让人持续盯住每一步。

放手运行前的四个门

任务是否可以与其他工作并行？错误是否能通过版本控制或快照恢复？执行权限是否被限制在明确范围？结束时是否有足够证据进行独立 review？四项任一为否，持续人工观察仍可能是必要控制。

11.3 把要求从记忆升级为机制

随着上下文增长，指令遵循会下降。最后的 prompt 和新加载的 Skill 常更容易获得注意，但这不是可靠记忆方案。课程建议把关键要求重新落地：短 checklist 放在阶段边界，Spec 与 Skill 保留目标和转向，类型、属性测试、CI gate 与权限沙箱提供机器约束，压缩摘要只保留目标、证据和未决问题。



图 35: Context 越长不等于记得越牢：把关键要求从文字升级成机制 (原视频画面：01:31:30–01:31:45)

11.4 本章小结

Token 只有在反馈能减少不确定性时才转化为知识。研究任务需要便宜、局部、可归因的实验；生产任务需要并行、可恢复的执行和独立验收；长上下文则需要机制化记忆。

12 在不确定性中搜索：算力、宽度与多样性

12.1 这次不是解题

封闭问题给定要求、路线和正确答案，可以沿一条思维链逐步逼近。开放研究没有唯一解，也没有预先知道的“已经够好”概率。课程用 Herbert Simon 的“满意化”提醒：搜索必须有停止条件，但停止不是证明找到了全局最优，而是在资源、证据和风险下接受当前方案。

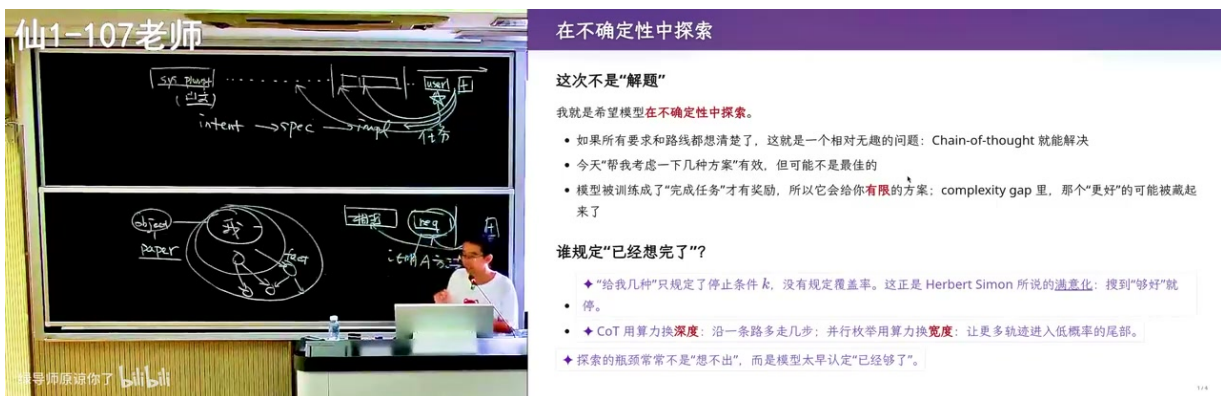


图 36: 开放任务不是解题：显式定义探索路线、停止条件与校验宽度 (原视频画面：01:33:15–01:33:30)

单条 chain-of-thought 用算力换深度；并行生成多条路线，再用独立校验筛选，则用算力换宽度。对于命名、设计、研究假设等任务，只问“给我几个名字”会得到模型偏好下的窄样本。更好的做法是先定义探索轴，再让不同 Agent 或不同种子覆盖组合空间，最后使用另一条路线检查重复、缺陷和隐含偏见。

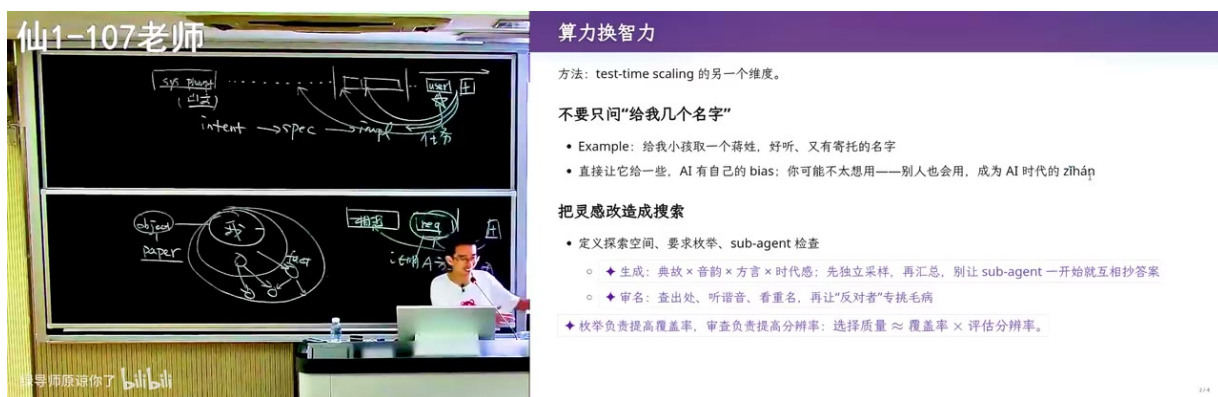


图 37: 把灵感生成改造成并行搜索: 先定义空间, 再采样和校验 (原视频画面: 01:34:30–01:34:45)

深度、宽度与校验的分工

深度用于沿一条路线解决依赖问题; 宽度用于覆盖互斥假设和风格; 校验用于淘汰看似新颖但不满足条件的候选。只增加深度会被早期假设锁定, 只增加宽度会产生大量噪声, 没有独立校验则无法把算力转成判断。

12.2 多采样不是多数投票

人的未来来自教育背景、生活经验和偏见的多样性。模型多采样若共享同一提示、同一资料和同一评估者, 可能只是同一偏见的细微抖动。课程借 Richard Sutton 的 Big World Hypothesis 强调, 真实世界远大于单个模型的已知结构; 群体智能不要求每个成员无偏, 而要求系统不只保留一种偏见, 并允许互相质疑。

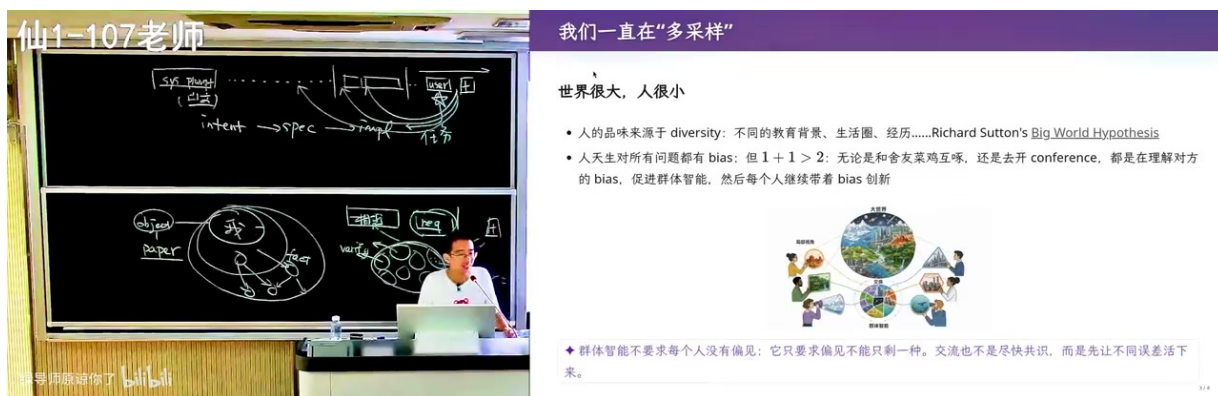


图 38: 通过不同生成路线与交叉批判保持群体多样性 (原视频画面: 01:36:45–01:37:00)

把灵感任务改造成搜索任务

先列探索维度和必须满足的约束; 让不同采样分别覆盖不同维度, 不许先互看答案; 用独立校验器查重、检查约束和寻找反例; 最后由人比较无法自动量化的意义、风险与长期后果。校验者与生成者若完全共享上下文, 应额外引入外部证据或人工复核。

12.3 墙内的超人，墙外的世界

结尾的“墙内超人”图像总结了课程立场：在规格和验证器围出的墙内，算力可以把搜索、实现和修复放大到超人的规模；墙外仍是开放世界，包含未写入上下文的人、制度、伦理、资源和不可逆后果。课程引用 Stanford AI Index 2025 中产业来源模型接近九成的统计，提出算力正在成为生产资料，但算力的所有者并不会自动获得正确目标。



图 39: 算力成为生产资料后, Agent 可以在墙内超人化, 人仍负责墙和外部世界(原视频画面: 01:37:30–01:37:45)

最终原则

能验证的，尽量写成机制；不能验证的，明确保留人的判断；有风险的，把边界放在模型之外；有不确定性的，用多路线搜索而不是伪装成确定答案。提示词工程的成熟标志，不是模型更听话，而是系统更诚实地知道自己完成了什么、没有完成什么。

12.4 本章小结

算力可以同时换取推理深度和探索宽度，但只有独立校验、真实证据与多样化路线才能把算力转成可靠判断。人最终负责定义搜索空间、停止条件和不可跨越的墙。

13 结语：把一句话升级为可验证系统

本讲从一个日常词“提示词”出发，最终抵达生成式软件工程的控制面。模型看到的是整个 context；context 里的约束决定搜索空间；Verifier 决定系统是否知道自己到达；Harness 和操作系统决定动作能否越界；反馈循环决定长程工作是否真正增加知识。

一份可直接使用的 Agent 任务骨架

1. Intent：为什么做，谁会使用，错误会伤害谁；
2. Spec：必须成立的行为、不可违反的边界、允许模型决定的自由度；
3. Evidence：事实来自哪里，何时获取，哪些仍是待核实假设；
4. Actions：允许使用哪些工具，哪些动作需批准，怎样回滚；

5. **Verifier**: 机器检查、独立 review、真实使用证据分别是什么;
6. **Stop**: 成功、显式失败和请求人工介入各自满足什么条件;
7. **Memory**: 哪些经验沉淀进 Skill, 哪些约束升级为测试、接口和 CI。

这套骨架也说明, 人类工作没有被一个更长 prompt 消灭。真正困难的是识别未写出的目标、选择值得搜索的问题、设计不易被投机的证据、控制真实世界权限, 并在证据不足时诚实地停下来。Agent 可以把墙内的执行能力推到极高, 但墙如何画、门何时开、墙外的人承担什么后果, 仍然是软件工程的核心。

课后练习

选一个你最近交给 AI 的真实任务, 不增加形容词, 只重写它的 Intent、Spec、Evidence、Actions、Verifier、Stop 和 Memory。让另一个人只凭这七项判断: 成功是否可识别, 失败是否可恢复, 权限是否过宽, 哪些价值仍需人工判断。再让 Agent 执行最小版本, 并记录第一次偏航来自哪一层。

资料状态说明

制作本讲义时, 课程主页可确认课程名称、教师与整体安排, 但未提供本讲可独立下载的官方课件或逐字字幕。因此, 画面内容以原视频为准, 字幕仅用于定位与语义互证; 所有可编辑源文件、图像清单和质量报告随讲义一并保存, 便于未来用官方材料更新。